

**SPEECH ENHANCEMENT USING RESIDUAL
CONVOLUTIONAL NEURAL NETWORK**

**A PROJECT SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF**

MASTER OF COMPUTER APPLICATIONS

BY

NAKKA PRAVEEN KUMAR

Roll No: 131723862017

Under The Supervision Of

Mrs. P. PREETI PAYAL

HOD, Department of MCA,

Princeton Pg College of Information and Technology,

Ramanthapur, Hyderabad.



FACULTY OF INFORMATICS

OSMANIA UNIVERSITY, HYDERABAD-

500007,

TELANGANA, INDIA.

August - 2025



CERTIFICATE

FACULTY OF INFORMATICS

This is to certify that, the following student with the name **NAKKA PRAVEEN KUMAR** bearing the **131723862017** pursuing **MCA** in **PRINCETON PG COLLEGE OF INFORMATION AND TECHNOLOGY, HYDERABAD**, has successfully completed the project **Speech Enhancement Using Residual Convolutional Neural Network** as a part of the curriculum in our college.

Head of Department

Internal Examiner

External Examiner

DECLARATION

I hereby declare that this Project Report titled **“SPEECH ENHANCEMENT USING RESIDUAL CONVOLUTIONAL NEURAL NETWORK”** submitted by me to the Department of Computer science, **PRINCETON PG COLLEGE OF IT** is not submitted elsewhere.

NAKKA PRAVEEN KUMAR

131723862017

ACKNOWLEDGEMENT

I would like to express my gratitude to our beloved principal, **Prof. M. Swathi Reddy**, for generously granting me permission to use the college's resources, which were essential for my projects.

With deep gratitude, I acknowledge **Mrs. P. Preeti Payal**, Head of the MCA Department, under whose valuable guidance and painstaking supervision I was able to successfully complete my project work.

I would also like to thank **Mrs. A. Sowjanya and Mrs. S. Swapna**, Faculty of the M.C.A. Department, for their valuable guidance and painstaking supervision, which enabled me to successfully complete my project work

NAKKA PRAVEEN KUMAR

131723862017

ABSTRACT

This paper deals with Deep Speech Enhancement which uses Deep Neural Networks to denoise a noisy speech to produce a clean speech thus removing almost all of the background noise. The method proposed here uses two dimensional convolutional neural networks with residual connections which take advantage of two key facts i.e the non linear functions learned by convolution neural network and the linearity introduced due to residual networks. These factors help remove noise even in low SNR conditions where usually most of the other speech enhancement models fail. The model proposed is fast, robust and lightweight and requires less training as compared to other deep learning models employed for the same task. The model is not expected to produce state of the art results but produces reasonable results keeping in mind the resource limitations and the small size of the model. The model is built upon previous research done in the field and main aim is the reduction in model size while simultaneously preserving the performance.

Keywords—Speech Enhancement, Speech Denoising, Convolutional Neural Network, SNR, Noise Reduction, Noise Distortion, STFT, ISTFT.

LIST OF CONTENTS

S. NO	TOPIC	PAGE.NO
1	1. INTRODUCTION	1-2
	1.1 SYSTEM STUDY	3
	1.1.1 FEASIBILITY STUDY	3
2	2. LITERATURE SURVEY	4-7
3	3. SYSTEM ANALYSIS	8
	3.1 EXISTING SYSTEM	8
	3.2 PROPOSED SYSTEM	9
	3.3 HARDWARE AND SOFTWARE REQUIREMENTS	10
	3.4 SYSTEM SPECIFICATION	11
	3.5 IMPLEMENTATION	12
	3.5.1 MODULES	12
4	4. DOMAIN DISCRPTION	13
	4.1 INTRODUCTION OF DEEP LEARNING	13
	4.2 WHAT IS DEEP LEATRNING	13
	4.3 ARTIFICIAL NEURAL NETWORK	14-16
5	5. SYSTEM DESIGN	17
	5.1 SYSTEM ARCHITECTURE	17
	5.2 DATA FLOW DIAGRAM	17
	5.3 UML DIAGRAM	18
	5.3.1 USECASE DIAGRAM	19
	5.3.2 CLASS DIAGRAM	20
	5.3.3 SEQUENCE DIAGRAM	21
	5.3.4 ACTIVITY DIAGRAM	22
6	6. SYSTEM IMPLEMENTATION	23
	6.1 PYTHON	23
	6.2 INTERACTIVE MODE PROGRAMMING	23
	6.3 SCRIPT MODE PROGRAMMING	23
	6.4 PYTHON IDENTIFIERS	24
	6.5 DJANGO	24-26
	6.6 SAMPLE CODE	26-31

LIST OF CONTENTS

7	7. SYSTEM TEST	32
	7.1 TYPES OF TESTS	32
	7.1.1 UNIT TESTING	32
	7.1.2 INTEGRATION TESTING	32
	7.1.3 FUNTIONAL TESTING	32
	7.1.4 SYSTEM TEST	33
	7.1.5 WHITE BOX TESTING	33
	7.1.6 BLACK BOX TESTING	33
	7.1.7 UNIT TESTING	33
	7.2 TEST STRATEGY AND APPROACH	33
	7.3 INTEGRATION TESTING	34
	7.4 ACCEPTANCE TESTING	34
	7.5 SAMPLE TEXT CASES	35
8	8. REPORTS	36-41
9	9. CONCLUSION	42
10	10. FUTURE ENHANCEMENT	43-44
11	11. REFERENCES	45
12	12. BASEPAPER	

LIST OF DIAGRAMS

S.NO	TOPIC	PAGE NO
1	5.1 SYSTEM ARCHITECTURE	17
2	5.2 DATA FLOW DIAGRAM	17
3	5.3.1 USE CASE DIAGRAM	19
4	5.3.2 CLASS DIAGRAM	20
5	5.3.3 SEQUENCE DIAGRAM	21
6	5.3.4 ACTIVITY DIAGRAM	22
7	5.3.5 ACTIVITY DIAGRAM	

LIST OF TABLES

TABLE.NO	TABLE NAME	
7.5	TEST CASES	35

1. INTRODUCTION

Speech enhancement has been long used as preprocessing step in various speech-related applications such as hearing aids [1, 2], speech coding [3], cochlea implants [4, 5] and automatic speech recognition [6]. Throughout the years extensive research has been carried out in this field starting from the basic techniques of Wiener filtering [7], Spectral Subtraction [8], non-negative matrix factorization [9]. These methods fail to learn any useful from the data but provide a blind estimation on predefined speech and noise models. Their model assumptions do not match real world complex non stationary noise, which leads to failure of these techniques in real world applications. Many deep neural networks have been explored in this domain showing promising result but almost all of them suffer the curse of complexity and resource usage. Traditionally, Fully Connected Deep Neural Networks were explored for the speech enhancement task but the size of neural network usually ends up in hundreds of megabytes which becomes difficult to use in embedded systems. The Convolutional Neural Networks however on the other hand consist of lesser number of parameters due to their weight sharing property. The models converge faster and are less resource exhaustive and can be embedded in portable devices. It has been seen that, historical models like Fully Connected Deep Neural Networks (DNN) use 20-30 times more parameters for same number of layers with same number of inputs, so convolutional layer based reduce parameters and decrease the training time to get to same results and also CNN works better in the case of finding local dependencies and leads to produce far better results than that of Dense networks. The paper attempts to find a memory efficient denoising algorithm that removes background noise from unseen test samples which include unseen speakers and unseen noise conditions. Unseen meaning the samples that were not used during the training of the network to ensure that the model generalizes well on real world conditions and does not stick to the training specific conditions. The paper consists of two convolutional neural network architectures on which the results are evaluated. The proposed architecture which is a convolutional neural network with skip connections and a similar model without the skip connections. Both these architectures are trained with and without dropouts to evaluate the performance measures and the proposed model without the dropouts outperforms other models based on the evaluation metrics Noise Reduction and Noise Distortion. Paper primarily focuses on these metrics derived from meansquare-error because we use this function to train our neural networks. Paper is divided into following sections. Section II contains problem description. Section III proposes a model to solve the formulated problem. Section IV contains various experiments that were carried out along with the setups of those experiments. Section V contains the results of the carried out experiments. Finally section VI contains the conclusions drawn.

1.1 SYSTEM STUDY

1.1.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are :

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

- **ECONOMICAL FEASIBILITY:**

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

- **TECHNICAL FEASIBILITY**

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

- **SOCIAL FEASIBILITY**

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

2. LITERATURE SURVEY

1) A Joint Bandwidth Expansion and Speech Enhancement Approach using Deep Neural Network

Authors : Taieba Taher, Nursadul Mamun, Md. Azad Hossain

Year : 2023.

Recently, joint bandwidth expansion and speech enhancement has been a topic of interest in the field of speech processing. The main challenge in this task is to increase the bandwidth of speech signals while enhancing their quality, simultaneously. Deep neural networks (DNNs) have shown great promise in addressing this challenge, as they can learn complex relationships between the input and output signals. In this study, a joint bandwidth expansion and speech enhancement approach using DNNs have been proposed, which is designed to simultaneously increase the bandwidth of speech signals and reduce noise, while preserving speech quality and intelligibility.

This approach leverages the capability of DNNs to simultaneously estimate the missing speech components and the noise profile in the degraded speech signal. The estimated speech components and the noise profile are then used to synthesize a full-band speech signal from a noisy signal with limited bandwidth with improved quality. The network employs three different phases such as oracle, imaged, and noisy phase along with the magnitude spectra to recover high band components. The joint approach demonstrates that the DNNbased bandwidth extension and speech enhancement can be effectively combined to produce high-quality speech signals, outperforms traditional speech enhancement methods, and offers promising solutions for various applications, including speech communication, speech recognition, and speech synthesis.

2) Speech Enhancement and Dereverberation with Diffusion-based Generative Models

Author : Julius Richter, Simon Welker, Jean-Marie

Lemercier, Timo Gerkmann

Year : 2023

In this work, we build upon our previous publication and use diffusion-based generative models for speech enhancement. We present a detailed overview of the diffusion process that is based on a stochastic differential equation and delve into an extensive theoretical examination of its

implications. Opposed to usual conditional generation tasks, we do not start the reverse process from pure Gaussian noise but from a mixture of noisy speech and Gaussian noise. This matches our forward process which moves from clean speech to noisy speech by including a drift term. We show that this procedure enables using only 30 diffusion steps to generate high-quality clean speech estimates. By adapting the network architecture, we are able to significantly improve the speech enhancement performance, indicating that the network, rather than the formalism, was the main limitation of our original approach. In an extensive cross-dataset evaluation, we show that the improved method can compete with recent discriminative models and achieves better generalization when evaluating on a different corpus than used for training. We complement the results with an instrumental evaluation using real-world noisy recordings and a listening experiment, in which our proposed method is rated best. Examining different sampler configurations for solving the reverse process allows us to balance the performance and computational speed of the proposed method. Moreover, we show that the proposed method is also suitable for dereverberation and thus not limited to additive background noise removal. Code and audio examples are available online¹.

3)Speech Enhancement Using Deep Learning Methods

Author : Asri Rizki Yuliani ^{a, *}, M. Faizal Amri ^b, Endang Suryawati ^a, Ade Ramdan ^a, Hilman F. Pardede ^a

Year : 2021

Speech enhancement, which aims to recover the clean speech of the corrupted signal, plays an important role in the digital speech signal processing. According to the type of degradation and noise in the speech signal, approaches to speech enhancement vary. Thus, the research topic remains challenging in practice, specifically when dealing with highly non-stationary noise and reverberation. Recent advance of deep learning technologies has provided great support for the progress in speech enhancement research field. Deep learning has been known to outperform the statistical model used in the conventional speech enhancement. Hence, it deserves a dedicated survey. In this review, we described the advantages and disadvantages of recent deep learning approaches. We also discussed challenges and trends of this field. From the reviewed works, we concluded that the trend of the deep learning architecture has shifted from the standard deep neural network (DNN) to convolutional neural network (CNN), which can efficiently learn temporal information of speech signal, and generative adversarial network (GAN), that utilize two networks training.

4)Speech Enhancement Using Residual Convolutional Neural Network**Author : Harinder Singh Mashiana, Abhishek Salaria, Kamaldeep Kaur****Year : 2019**

Because of its widespread use in facial analysis tasks, automatic gender and age prediction from facial photos has attracted a lot of attention. Because of the wide intra-class diversity in face images—including variations in lighting, position, size, and occlusion—current models frequently fail to achieve accuracy. These problems make it hard to use them in practical situations. To accurately predict gender and age group, we present a deep learning architecture that merges attentional and residual convolutional networks in this study. Our model is able to improve prediction accuracy by zeroing in on important and relevant areas of the face thanks to the attention mechanism. We use a multi-task learning strategy to enhance the accuracy of age prediction by adding the projected gender to the age classifier's feature embeddings. We have trained our model using a popular dataset that includes information about the age and gender of faces, and the results are encouraging. We also demonstrate that the trained model has learnt to focus on the important facial features—the eyes and the mouth, for example—by visualizing its attention maps; these features are critical for precise age and gender categorization. For practical uses involving face recognition, this method offers a solid answer.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

Traditional Methods: Early techniques like Wiener filtering, spectral subtraction, and non-negative matrix factorization are mentioned. These methods provide basic denoising but fail in real-world conditions with non-stationary noise.

Fully Connected Deep Neural Networks (DNN): DNNs were initially explored for speech enhancement, but these models are resource-intensive with large model sizes (100-150 MB). They tend to have higher training times and are impractical for embedded systems

Generative Adversarial Networks (SEGAN): SEGAN-based models reduce the size to around 40-60 MB while enhancing speech quality. However, they are still relatively large for portable devices

Convolutional Autoencoders: These models reduce the model size significantly (10-20 MB) by using convolutional layers but still leave room for improvement in terms of real-time performance on embedded systems

3.2 PROPOSED SYSTEM:

The proposed system in this paper is a Residual Convolutional Neural Network (RCNN) that builds upon the efficiency of CNNs but incorporates skip connections to preserve important features and mitigate the vanishing gradient problem.

The RCNN aims to:

Reduce Model Size: The RCNN is extremely lightweight (0.6-0.7 MB) compared to other models.

Improve Noise Reduction: It achieves a noise reduction of 96.9% while keeping speech distortion low (4.2%).

Skip Connections: These connections allow gradients to move deeper into the network, improving learning without increasing the model's complexity(mashiana2019 (2)).

Algorithms:

Residual CNN (RCNN): The core of the proposed system, it uses convolutional layers without pooling and features skip connections to enhance the model's ability to preserve useful features throughout the network.
int,float,complex,bool,string,list,tuple,set,dictionary,range,byte,bytearray,frozen set.

3.3 HARDWARE AND SOFTWARE REQUIREMENTS

3.3.1 REQUIREMENT ANALYSIS

The project involved analyzing the design of few applications so as to make the application more users friendly. To do so, it was really important to keep the navigations from one screen to the other well ordered and at the same time reducing the amount of typing the user needs to do. In order to make the application more accessible, the browser version had to be chosen so that it is compatible with most of the Browsers.

3.3.2 REQUIREMENT SPECIFICATION

Functional Requirements

- ❖ Graphical User interface with the User.

Software Requirements

For developing the application the following are the Software Requirements:

1. Python
2. Django

Operating Systems supported

1. Windows 10 64 bit OS

Technologies and Languages used to Develop

1. Python

Debugger and Emulator

- Any Browser (Particularly Chrome)

Hardware Requirements

- For developing the applications the following are the Hardware Requirements:
 - Processor : Intel i9
 - RAM : 32 GB
 - Space on Hard Disk : 1 TB (Minimum)

3.4 SYSTEM SPECIFICATION:

3.4.1 HARDWARE REQUIREMENTS:

- ❖ **System** : Intel i3
- ❖ **Hard Disk** : 1 TB.
- ❖ **Monitor** : 14' Colour Monitor.
- ❖ **Mouse** : Optical Mouse.
- ❖ **Ram** : 4GB.

3.4.2 SOFTWARE REQUIREMENTS:

- ❖ **Operating system** : Windows 10.
- ❖ **Coding Language** : Python.
- ❖ **Front-End** : Html. CSS
- ❖ **Designing** : Html ,css, javascript.
- ❖ **Data Base** : SQLite.

3.5 MODULES:

- User
- Admin
- Data collection
- CNN

MODULES DESCRIPTION:

User:

The User can register the first. While registering he required a valid user email and mobile for further communications. Once the user register then admin can activate the user. Once admin activated the user then user can login into our system. User can upload the dataset based on our dataset column matched. For algorithm execution data must be in float format. Here we took heavy vehicles related fuel consumption dataset. User can also add the new data for existing dataset based on our Django application. User can click the training in the web page so that the data calculated model results like(mean_absolute error, mean_square error, r2_error) based on the algorithms. User can display the prediction results. After that user can logout.

Admin:

Admin can login with his login details. Admin can activate the registered users. Once he activate then only the user can login into our system. Admin can view the overall data in the browser. Admin can click the model Results in the web page so calculated (mean_absolute error, mean_square error, r2_error) based on the algorithms. After that admin can logout.

Data collection:

The model is developed by using duty cycles collected from a single truck, with an approximate mass of 8,700 kg exposed to a variety of transients including both urban and highway traffic in the Indianapolis area. Data was collected using the SAE J1939 standard for serial control and communications in heavy duty vehicle networks [24]. Twelve drivers were asked to exhibit good or bad behavior over two different routes. Drivers exhibiting good behavior anticipated braking and allowed the vehicle to coast when possible. Some drivers participated more than others and as a result the distribution of drivers and routes is not uniform across the data set. This field test generated 3,302,890 data points sampled at 50 Hz from the vehicle CAN bus and a total distance of 778.89 km over 56 trips with varying distances. Most of the trips covered a distance of 10 km to 15 km. In order to

increase the number of data points, synthetic duty cycles over an extended distance were obtained by assembling segments from the field duty cycles selected at random. Moreover, a set of drivers are assigned to the training segments and a different set of drivers are assigned to the testing segments, thereby ensuring that the training (Ftr) and testing (Fts) data sets derived from the respective segments are completely separate.

CNN(Convolutional Neural Network):

The proposed model can easily be developed and deployed for each individual vehicle in a fleet in order to optimize fuel consumption over the entire fleet. The predictors of the model are aggregated over fixed window sizes of distance traveled. Different window sizes are evaluated and the results show that a 1 km window is able to predict fuel consumption with a 0.91 coefficient of determination and mean absolute peak-to-peak percent error less than 4% for routes that include both city and highway duty cycle segments.

4. DOMAIN DESCRIPTION

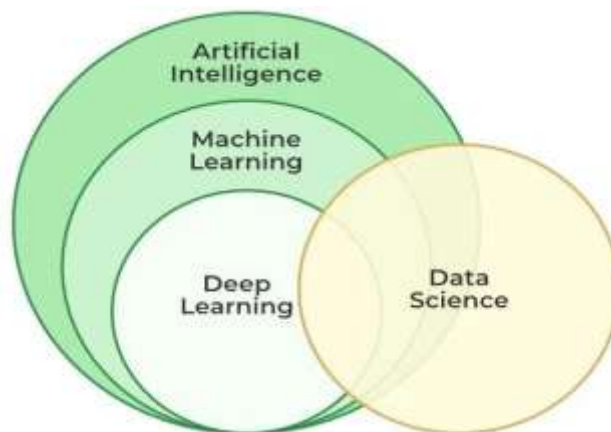
4.1 Introduction to Deep Learning

In the fast-evolving era of artificial intelligence, Deep Learning stands as a cornerstone technology, revolutionizing how machines understand, learn, and interact with complex data. At its essence, Deep Learning AI mimics the intricate neural networks of the human brain, enabling computers to autonomously discover patterns and make decisions from vast amounts of unstructured data. This transformative field has propelled breakthroughs across various domains, from computer vision and natural language processing to healthcare diagnostics and autonomous driving.

4.2 What is Deep Learning?

The definition of Deep learning is that it is the branch of [machine learning](#) that is based on artificial neural network architecture. An artificial neural network or [ANN](#) uses layers of interconnected nodes called neurons that work together to process and learn from the input data.

In a fully connected Deep neural network, there is an input layer and one or more hidden layers connected one after the other. Each neuron receives input from the previous layer neurons or the input layer. The output of one neuron becomes the input to other neurons in the next layer of the network, and this process continues until the final layer produces the output of the network. The layers of the neural network transform the input data through a series of nonlinear transformations, allowing the network to learn complex representations of the input data.



Today Deep learning AI has become one of the most popular and visible areas of machine learning, due to its success in a variety of applications, such as computer vision, natural language processing, and Reinforcement learning.

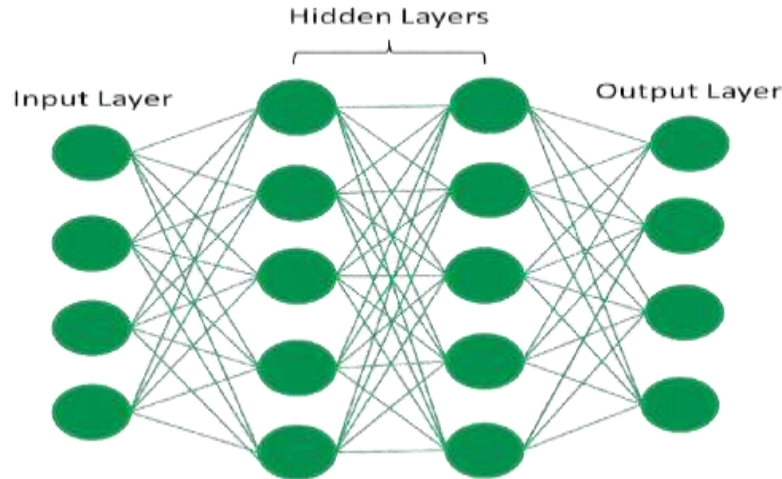
Deep learning AI can be used for supervised, unsupervised as well as reinforcement machine learning. it uses a variety of ways to process these.

- **Supervised Machine Learning:** [Supervised machine learning](#) is the [machine learning](#) technique in which the neural network learns to make predictions or classify data based on the labeled datasets. Here we input both input features along with the target variables. the neural network learns to make predictions based on the cost or error that comes from the difference between the predicted and the actual target, this process is known as backpropagation. Deep learning algorithms like Convolutional neural networks, Recurrent neural networks are used for many supervised tasks like image classifications and recognition, sentiment analysis, language translations, etc.
- **Unsupervised Machine Learning:** [Unsupervised machine learning](#) is the [machine learning](#) technique in which the neural network learns to discover the patterns or to cluster the dataset based on unlabeled datasets. Here there are no target variables. while the machine has to self-determined the hidden patterns or relationships within the datasets. Deep learning algorithms like autoencoders and generative models are used for unsupervised tasks like clustering, dimensionality reduction, and anomaly detection.
- **Reinforcement Machine Learning:** [Reinforcement Machine Learning](#) is the [machine learning](#) technique in which an agent learns to make decisions in an environment to maximize a reward signal. The agent interacts with the environment by taking action and observing the resulting rewards. Deep learning can be used to learn policies, or a set of actions, that maximizes the cumulative reward over time. Deep reinforcement learning algorithms like Deep Q networks and Deep Deterministic Policy Gradient (DDPG) are used to reinforce tasks like robotics and game playing etc.

4.3 Artificial neural networks

[Artificial neural networks](#) are built on the principles of the structure and operation of human neurons. It is also known as neural networks or neural nets. An artificial neural network's input layer, which is the first layer, receives input from external sources and passes it on to the hidden layer, which is the second layer. Each neuron in the hidden layer gets information from the neurons in the previous layer, computes the weighted total, and then transfers it to the neurons in the next layer. These connections are weighted, which

means that the impacts of the inputs from the preceding layer are more or less optimized by giving each input a distinct weight. These weights are then adjusted during the training process to enhance the performance of the model.



Artificial neurons, also known as units, are found in artificial neural networks. The whole Artificial Neural Network is composed of these artificial neurons, which are arranged in a series of layers. The complexities of neural networks will depend on the complexities of the underlying patterns in the dataset whether a layer has a dozen units or millions of units. Commonly, Artificial Neural Network has an input layer, an output layer as well as hidden layers. The input layer receives data from the outside world which the neural network needs to analyze or learn about.

4.3.1 Types of neural networks

Deep Learning models are able to automatically learn features from the data, which makes them well-suited for tasks such as image recognition, speech recognition, and natural language processing. The most widely used architectures in deep learning are feedforward neural networks, convolutional neural networks (CNNs), and recurrent neural networks (RNNs).

1. [Feedforward neural networks \(FNNs\)](#) are the simplest type of ANN, with a linear flow of information through the network. FNNs have been widely used for tasks such as image classification, speech recognition, and natural language processing.

2. [Convolutional Neural Networks \(CNNs\)](#) are specifically for image and video recognition tasks. CNNs are able to automatically learn features from the images, which makes them well-suited for tasks such as image classification, object detection, and image segmentation.
3. [Recurrent Neural Networks \(RNNs\)](#) are a type of neural network that is able to process sequential data, such as time series and natural language. RNNs are able to maintain an internal state that captures information about the previous inputs, which makes them well-suited for tasks such as speech recognition, natural language processing, and language translation.

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE:

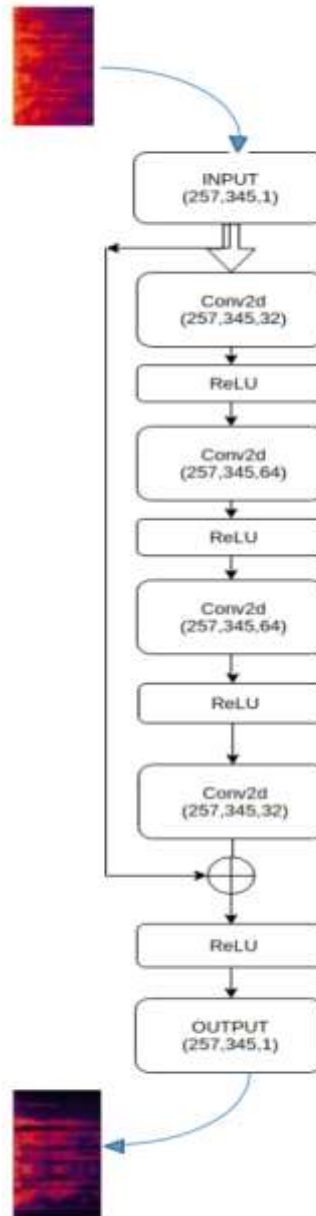


Fig 5.1 System Architecture

5.2 DATA FLOW DIAGRAM:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.

2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

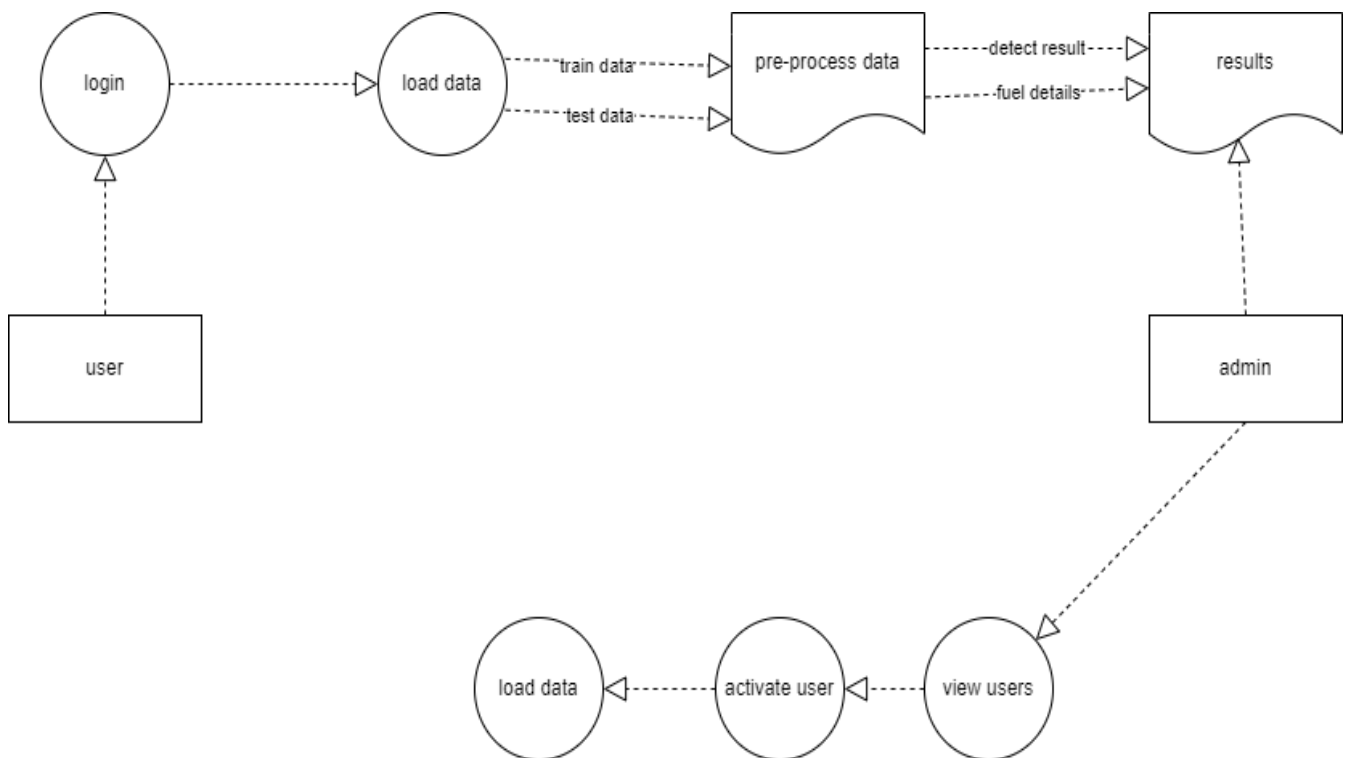


Fig 5.2 Data Flow Diagram

5.3 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

5.3.1 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

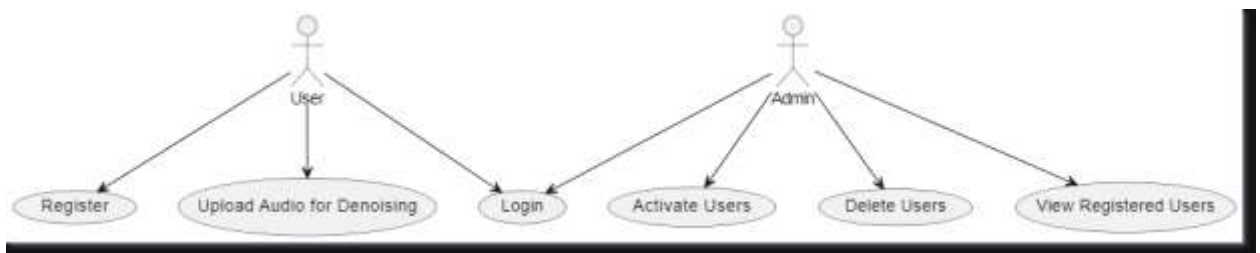


Fig - 5.3.1 Use Case Diagram

5.3.2 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

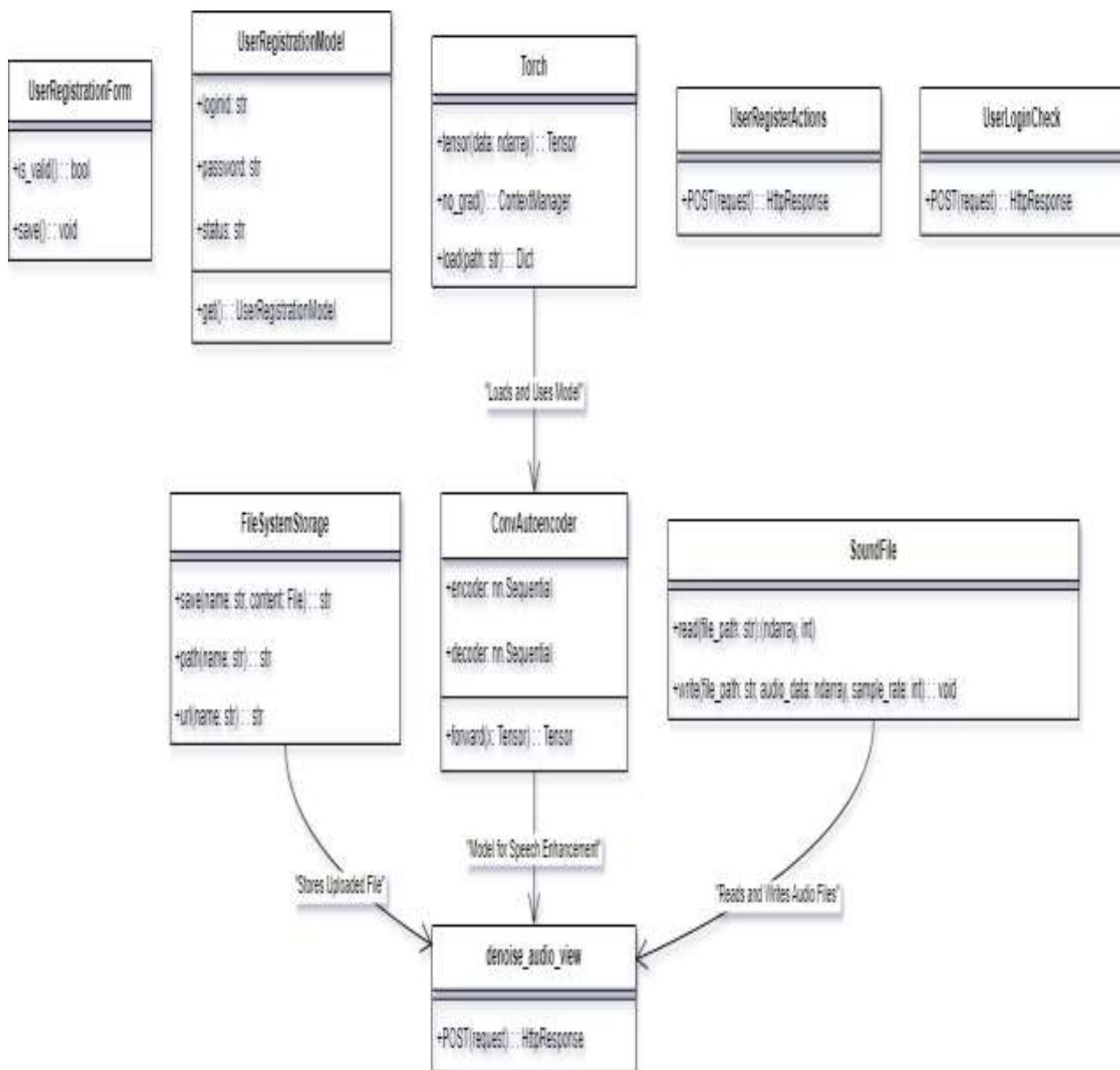


Fig - 5.3.2 Class Diagram

5.3.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

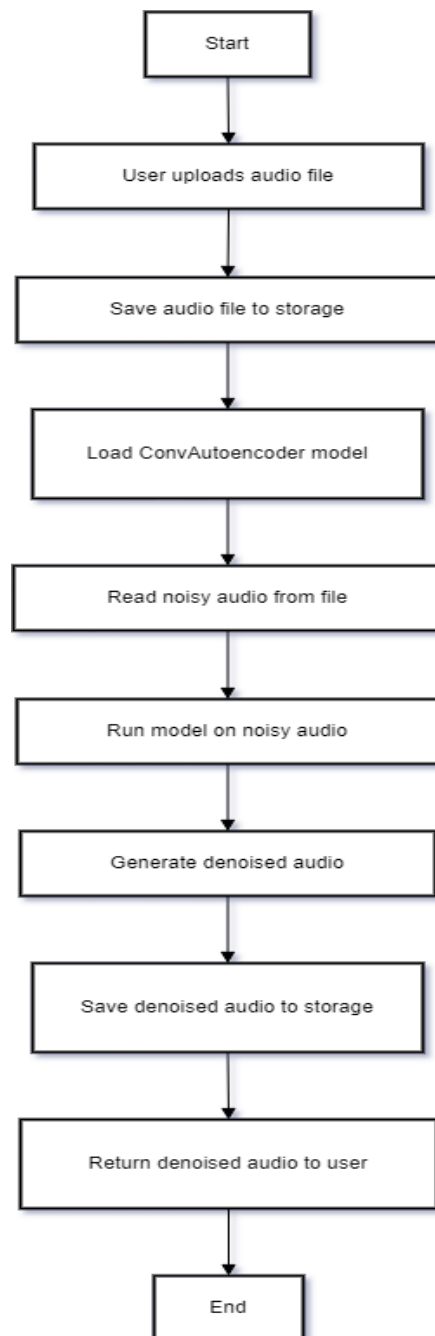


Fig - 5.3.3 Sequence Diagram

5.3.4 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

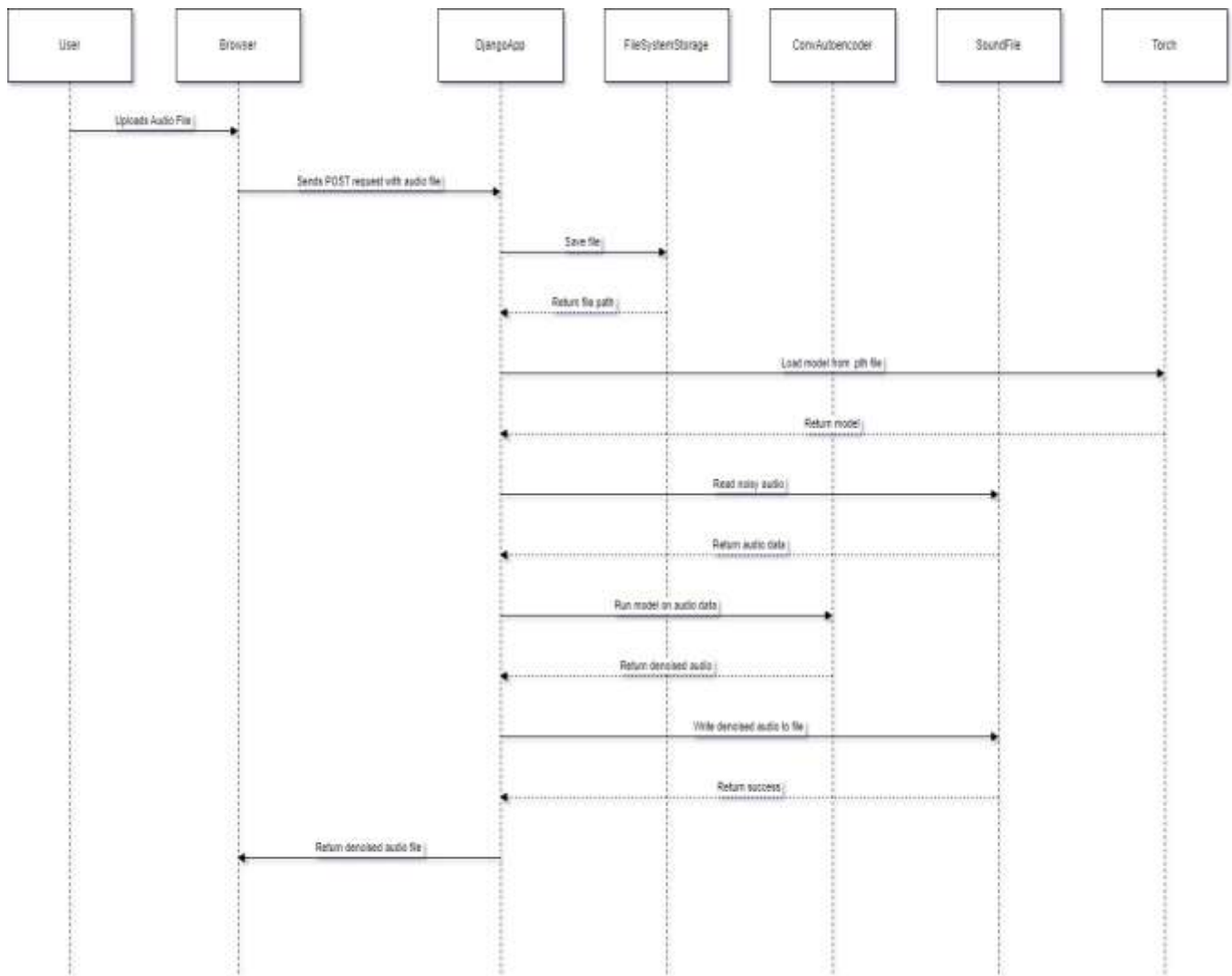


Fig - 5.3.4 Activity Diagram

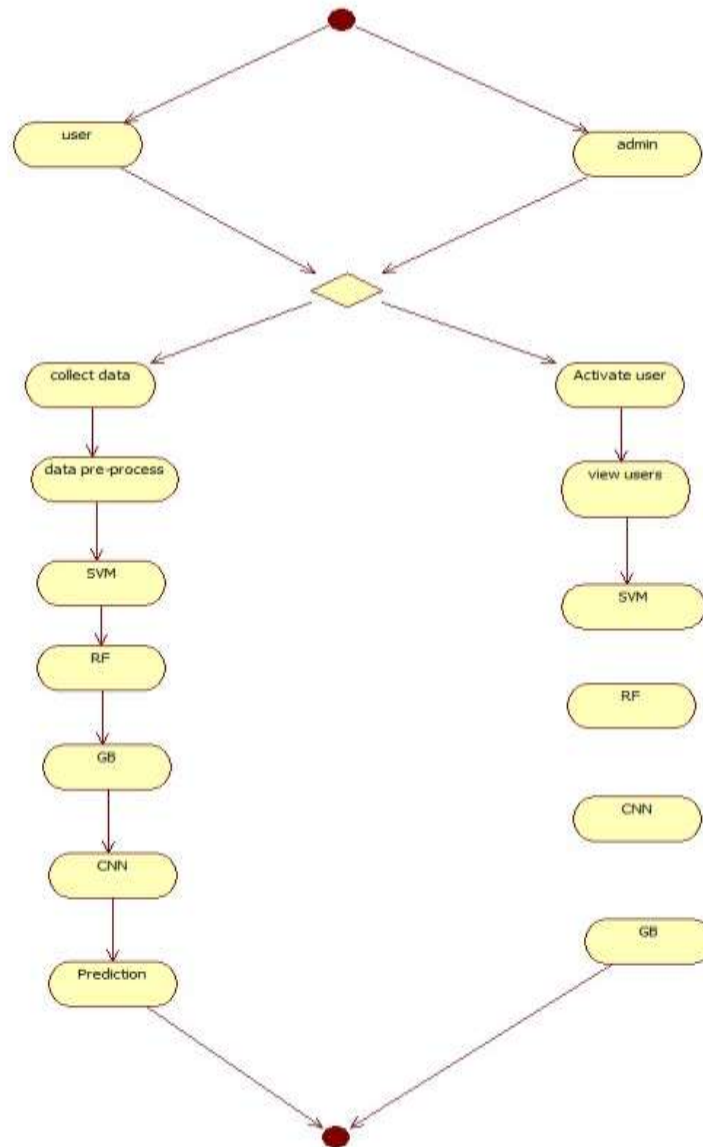


Fig - 5.3.5 Activity Diagram

6. SYSTEM IMPLEMENTATION

6.1 PYTHON

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An [interpreted language](#), Python has a design philosophy that emphasizes code [readability](#) (notably using [whitespace](#) indentation to delimit [code blocks](#) rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer [lines of code](#) than might be used in languages such as [C++](#) or [Java](#). It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many [operating systems](#). [CPython](#), the [reference implementation](#) of Python, is [open source](#) software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit [Python Software Foundation](#). Python features a [dynamic type](#) system and automatic [memory management](#). It supports multiple [programming paradigms](#), including [object-oriented](#), [imperative](#), [functional](#) and [procedural](#), and has a large and comprehensive [standard library](#).

6.2 Interactive Mode Programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt –
\$ python

Python 2.4.3 (#1, Nov 11 2010, 13:34:43)

[GCC 4.1.2 20080704 (Red Hat 4.1.2-48)] on linux2

Type "help", "copyright", "credits" or "license" for more information.

>>>

Type the following text at the Python prompt and press the Enter –

>>> print "Hello, Python!"

If you are running new version of Python, then you would need to use print statement with parenthesis as in print ("Hello, Python!");. However in Python version 2.4.3, this produces the following result –

Hello, Python!

6.3 Script Mode Programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.

Let us write a simple Python program in a script. Python files have extension .py. Type the following source code in a test.py file – Live Demo

```
print "Hello, Python!"
```

We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows

```
$ python test.py
```

This produces the following result – Hello, Python!

Let us try another way to execute a Python script. Here is the modified test.py file –

Live Demo

```
#!/usr/bin/python
```

```
print "Hello,
```

```
Python!"
```

We assume that you have Python interpreter available in /usr/bin directory. Now, try to run this program as follows –

```
$ chmod +x test.py    # This is to make file executable
```

```
$/test.py
```

This produces the following result –

Hello, Python!

6.4 Python Identifiers

A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (`_`) followed by zero or more letters, underscores and digits (0 to 9).

Python does not allow punctuation characters such as `@`, `$`, and `%` within identifiers. Python is a case sensitive programming language. Thus, Manpower and manpower are two different identifiers in Python. Here are naming conventions for Python identifiers –

Class names start with an uppercase letter. All other identifiers start with a lowercase letter. Starting an identifier with a single leading underscore indicates that the identifier is private. Starting an identifier with two leading underscores indicates a strongly private identifier.

If the identifier also ends with two trailing underscores, the identifier is a language-defined special name.

6.5 Django

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source. Django's primary goal is to ease the creation of complex, database-driven websites.

Django emphasizes [reusability](#) and "pluggability" of components, rapid development, and the principle of [don't repeat yourself](#). Python is used throughout, even for settings files and data models.

6.5.1 Features of Django:

1. **Full-Featured Framework:** Django comes with a wide range of built-in features like authentication, admin interface, and ORM, reducing the need for third-party tools.
2. **MTV Architecture:** It follows the Model-Template-View pattern, promoting clear separation between business logic, presentation, and data handling.
3. **Admin Interface:** Automatically generates a powerful admin dashboard for managing models and application data.
4. **ORM Integration:** Includes a built-in Object-Relational Mapper to interact with databases using Python code instead of raw SQL.
5. **Security:** Provides built-in protection against common web attacks such as CSRF, XSS, and SQL injection.
6. **Scalability:** Designed to handle large-scale applications efficiently through modular and reusable components.
7. **URL Routing:** Uses clean and readable URL patterns that are easy to manage and customize.
8. **Templating System:** Integrates a robust templating engine to generate dynamic HTML content efficiently.
9. **Middleware Support:** Offers middleware layers to process requests/responses globally for logging, authentication, and more.
10. **Comprehensive Documentation:** Comes with detailed and well-maintained documentation, ideal for developers at all levels.

6.5.2 Django Implementation diagram

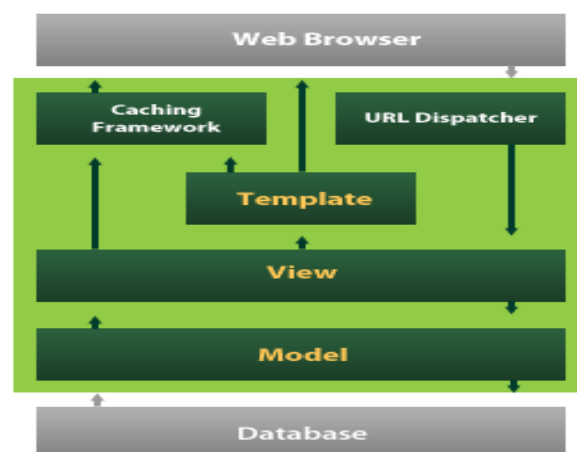


Fig- 6.5.2 Flask Implementation diagram

Basic Syntax of Django:

```
myproject/  
manage.py  
myproject/  
__init__.py  
settings.py  
urls.py  
wsgi.py  
The Project Structure
```

6.6 Sample code(not change) :

Userside views:

```
from django.shortcuts import render, HttpResponseRedirect  
from django.contrib import messages  
from .forms import UserRegistrationForm  
from .models import UserRegistrationModel, VideoAnalysis  
import numpy as np  
import pandas as pd  
import os  
import torch  
import torch.nn as nn  
from sklearn.metrics import roc_auc_score, recall_score, precision_score, f1_score, accuracy_score  
from sklearn.model_selection import RandomizedSearchCV, cross_val_score, KFold, train_test_split  
from scipy.stats import uniform, randint  
from sklearn.metrics import matthews_corrcoef  
from sklearn.metrics import average_precision_score  
from sklearn.metrics import f1_score  
import xgboost  
from xgboost import XGBClassifier  
from django.conf import settings  
from django.shortcuts import render
```

```
from django.core.files.storage import FileSystemStorage
from django.conf import settings
import matplotlib.pyplot as plt
import numpy as np

# from django.conf import settings
# from .models import predictions
# import os
# import pandas as pd

# Create your views here.
def UserRegisterActions(request):
    if request.method == 'POST':
        form = UserRegistrationForm(request.POST)
        if form.is_valid():
            print('Data is Valid')
            form.save()
            messages.success(request, 'You have been successfully registered')
            form = UserRegistrationForm()
            return render(request, 'UserRegistrations.html', {'form': form})
        else:
            messages.success(request, 'Email or Mobile Already Existed')
            print("Invalid form")
        else:
            form = UserRegistrationForm()
    return render(request, 'UserRegistrations.html', {'form': form})

def UserLoginCheck(request):
    if request.method == "POST":
        loginid = request.POST.get('loginid')
        pswd = request.POST.get('pswd')
        print("Login ID = ", loginid, ' Password = ', pswd)
        try:
            check = UserRegistrationModel.objects.get(loginid=loginid, password=pswd)
            status = check.status
```

```
print('Status is = ', status)
    if status == "activated":
        request.session['id'] = check.id
        request.session['loggeduser'] = check.name
        request.session['loginid'] = loginid
        request.session['email'] = check.email
        print("User id At", check.id, status)
        return render(request, 'users/UserHomePage.html', {})
    else:
        messages.success(request, 'Your Account Not at activated')
        return render(request, 'UserLogin.html')
except Exception as e:
    print('Exception is ', str(e))
    pass
    messages.success(request, 'Invalid Login id and password')
return render(request, 'UserLogin.html', {})

def UserHome(request):
    return render(request, 'users/UserHomePage.html', {})

import os
import torch
import numpy as np
import soundfile as sf
from django.http import JsonResponse
from django.views import View
from django.core.files.storage import FileSystemStorage

from django.shortcuts import render
from django.core.files.storage import FileSystemStorage
import torch
import soundfile as sf
import os
from django.conf import settings
```

```
import torch
import torch.nn as nn

class ConvAutoencoder(nn.Module):
    def __init__(self):
        super(ConvAutoencoder, self).__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Conv1d(1, 16, kernel_size=9, padding=4), # Output: [16, T]
            nn.ReLU(),
            nn.Conv1d(16, 32, kernel_size=9, padding=4), # Output: [32, T]
            nn.ReLU(),
            nn.Conv1d(32, 64, kernel_size=9, padding=4), # Output: [64, T]
            nn.ReLU(),
        )

        # Decoder
        self.decoder = nn.Sequential(
            nn.Conv1d(64, 32, kernel_size=9, padding=4), # Output: [32, T]
            nn.ReLU(),
            nn.Conv1d(32, 16, kernel_size=9, padding=4), # Output: [16, T]
            nn.ReLU(),
            nn.Conv1d(16, 1, kernel_size=9, padding=4), # Output: [1, T]
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

model_path = settings.MEDIA_ROOT + '/' + 'cae_speech_enhancement.pth'
from django.shortcuts import render
from django.core.files.storage import FileSystemStorage
from django.conf import settings
```

```
import soundfile as sf
import torch

# Ensure this import is correct

from django.shortcuts import render
from django.core.files.storage import FileSystemStorage
from django.conf import settings
import soundfile as sf
import torch
import matplotlib.pyplot as plt
import numpy as np
import os
# Ensure this import is correct

def denoise_audio_view(request):
    context = {}

    if request.method == 'POST' and 'audio_file' in request.FILES:
        audio_file = request.FILES['audio_file']
        fs = FileSystemStorage()
        filename = fs.save(audio_file.name, audio_file)
        file_path = fs.path(filename)

        # Load your model
        model = ConvAutoencoder()
        # Adjust this path
        model.load_state_dict(torch.load(model_path, map_location=torch.device('cpu')))
        model.eval()

        # Read the noisy audio
        noisy_audio, sample_rate = sf.read(file_path)

        # Denoise the audio
```

```
denoised_audio = denoise_audio(noisy_audio, model)

# Save the denoised audio
output_filename = 'denoised_' + audio_file.name

output_path = fs.path(output_filename)
sf.write(output_path, denoised_audio, sample_rate)

# Create and save waveform plots
noisy_plot_path = create_waveform_plot(noisy_audio, 'Noisy Audio Waveform',
'noisy_waveform.png')
denoised_plot_path = create_waveform_plot(denoised_audio, 'Denoised Audio Waveform',
'denoised_waveform.png')

# Set the URLs for both noisy and denoised audio and their waveforms
context['noisy_audio_url'] = fs.url(filename)
context['denoised_audio_url'] = fs.url(output_filename)
context['noisy_plot_url'] = fs.url(os.path.basename(noisy_plot_path))
context['denoised_plot_url'] = fs.url(os.path.basename(denoised_plot_path))

return render(request, 'users/results.html', context)

def denoise_audio(noisy_audio, model):
    with torch.no_grad():
        noisy_tensor = torch.tensor(noisy_audio).unsqueeze(0).float()
        denoised_tensor = model(noisy_tensor)
        denoised_audio = denoised_tensor.squeeze(0).numpy()
    return denoised_audio

def create_waveform_plot(audio, title, filename):
    plt.figure(figsize=(10, 4))
    plt.plot(audio)
    plt.title(title)
    plt.xlabel('Sample')
```

```
plt.ylabel('Amplitude')
```

```
# Save the plot
```

```
fs = FileSystemStorage()
```

```
plot_path = fs.path(filename)
```

```
plt.savefig(plot_path)
```

```
plt.close()
```

```
return plot_path
```

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTING :

7.1.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

7.1.2 Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

7.1.3 Functional testing

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- **Valid Input** : identified classes of valid input must be accepted.
- **Invalid Input** : identified classes of invalid input must be rejected.
- **Functions** : identified functions must be exercised.
- **Output** : identified classes of application outputs must be exercised.
- **Systems/Procedures** : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

7.1.4 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

7.1.5 White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

7.1.6 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

7.1.7 Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

7.2 Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

7.2.1 Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

7.2.2 Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

7.3 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

7.3.1 Test Results: All the test cases mentioned above passed successfully. No defects encountered.

7.4 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

7.4.1 Test Results:

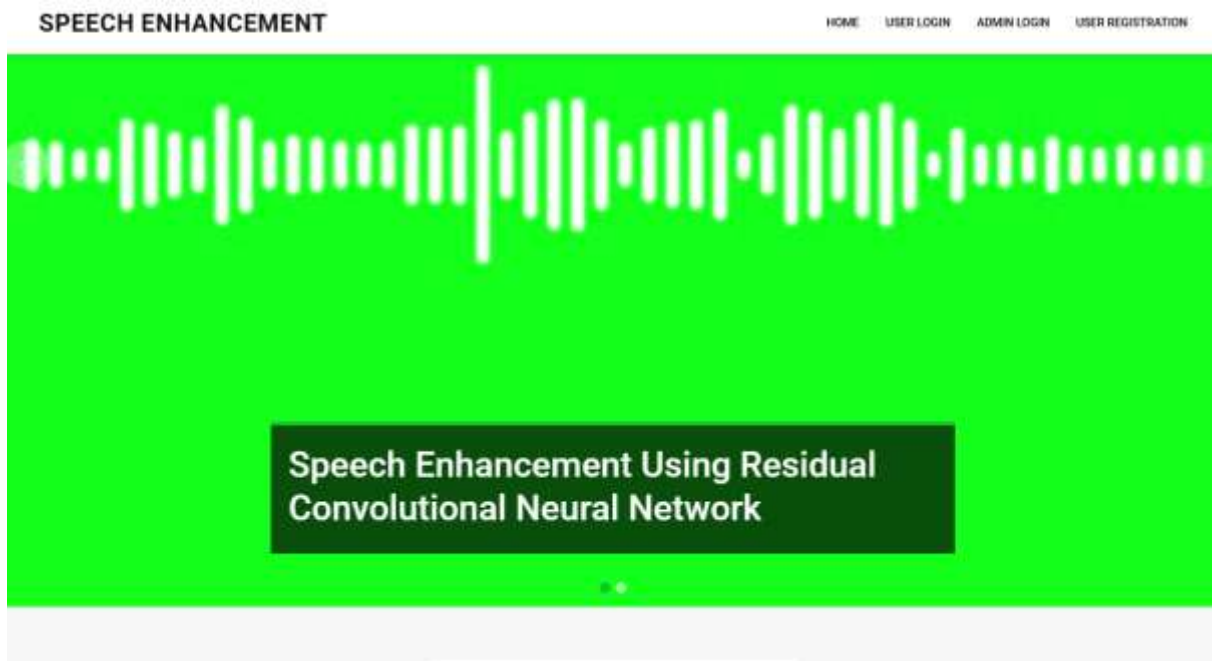
All the test cases mentioned above passed successfully. No defects encountered.

7.5 Test Cases:

S.no	Test Case	Excepted Result	Result	Remarks(IF Fails)
1	User Register	If User registration successfully.	Pass	If already user email exist then it fails.
2	User Login	If Username and password is correct then it will getting valid page.	Pass	Un Register Users will not logged in.
3	cnn	The request will be accepted by the cnn	Pass	The request will be not accepted by the cnn otherwise its failed
4	autoencoder	The request will be accepted by the autoencoder	Pass	Results not displayed otherwise its failed
5	rcnn	The request will be accepted by the rcnn	Pass	Results not displayed otherwise its failed
6	Model results	Display mean _square error, mean absolute error, r2_score	pass	Results not displayed otherwise its failed
7	View dataset by user	Data set will be displayed by the user	Pass	Results not true failed
8	User regression	Display reviews with true results	Pass	Results not true failed
9	Admin login	Admin can login with his login credential. If success he get his home page	Pass	Invalid login details will not allowed here
10	Admin can activate the register users	Admin can activate the register user id	Pass	If user id not found then it won't login.

8. REPORTS

HOME PAGE



USER REGISTRATION:

The screenshot shows the "User Registration Form" page. The page has a blue header with the title "SPEECH ENHANCEMENT" and navigation links: HOME, USER LOGIN, ADMIN LOGIN, and USER REGISTRATION. The main content area is white and contains a form titled "User Registration Form". The form includes the following fields: User Name, Login ID, Password, Mobile, Email, Locality, Address, City, State, and Status. A blue "Register" button is located at the bottom of the form.

ADMIN LOGIN:

The screenshot shows the Admin Login page. At the top, there is a navigation bar with the title "SPEECH ENHANCEMENT" on the left and links for "HOME", "USER LOGIN", "ADMIN LOGIN", and "USER REGISTRATION" on the right. Below the navigation bar is a large blue banner with the text "Speech Enhancement Using Residual Convolutional Neural Network". Underneath the banner is a white box titled "Admin Login Form". Inside this box, there are two input fields: "Login ID" with a placeholder "Enter Login ID" and "Pysword" with a placeholder "Enter Password". Below these fields are two buttons: a blue "Login" button and a grey "Reset" button.

ADMIN HOME :



USER DETAILS:

SPEECH ENHANCEMENT USING RESIDUAL CONVOLUTIONAL NEURAL NETWORK

HOME USER DETAILS LOGOUT

Speech Enhancement Using Residual Convolutional Neural Network

View Registered Users

S.No	Name	Login ID	Mobile	Email	Locality	Status	Activate	Delete
1	alex	alex	7780507824	alex@gmail.com	hyd	activated	Activated	Delete

USER LOGIN:

SPEECH ENHANCEMENT

HOME USER LOGIN ADMIN LOGIN USER REGISTRATION

Speech Enhancement Using Residual Convolutional Neural Network

User Login Form

Login ID

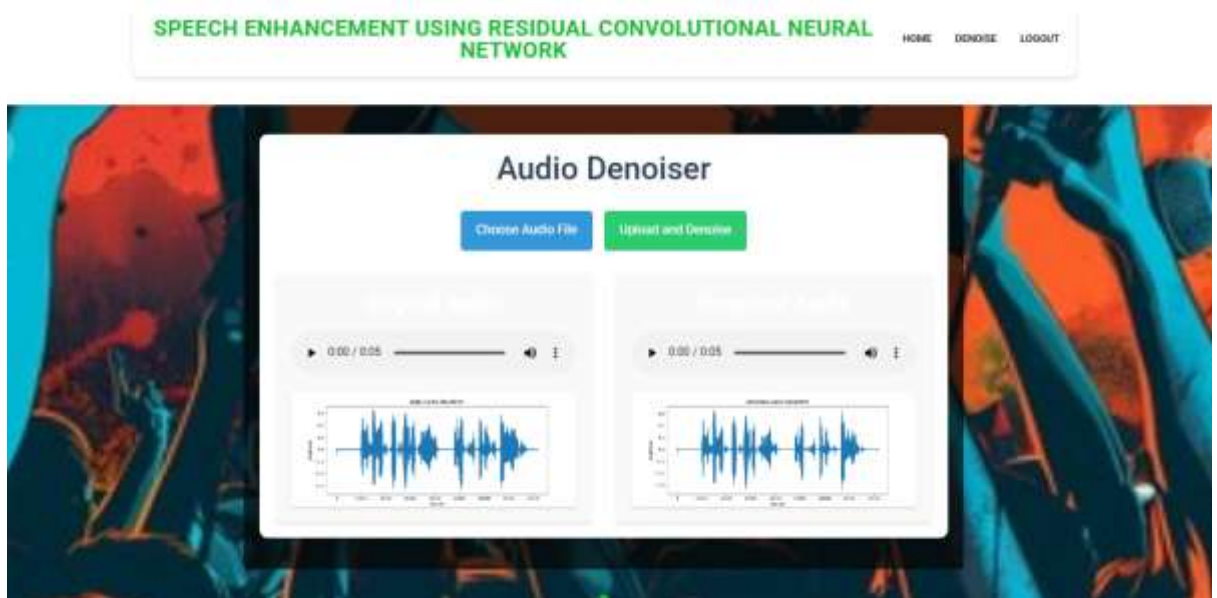
Password

[Login](#) [Reset](#)

USER HOME:



PREDICTION RESULT:



9. CONCLUSION

The focus of this paper is speech enhancement that is solved through the use of the neural networks. The work here focuses on a neural network architecture that is easy to implement, train and use in real world thus eliminating the major drawbacks of the neural networks. Proposed model reduces noise by 96.9% in the speech and produces speech distortion of 4.2%. We see 99.4% decrease in model size from the best performing model and only a decrease of 10% in performance. This study serves as a pioneering work for exploiting the full potential of neural networks without focusing on increasing their size for enhanced performance. In the future work, more noise types and SNR levels can be considered to build a more robust network

10. FUTURE ENHANCEMENT

To enhance the speech enhancement system in the paper, several improvements can be made. First, incorporating **phase-aware models** can improve speech reconstruction by modeling both magnitude and phase information, which the current model lacks. Adding **attention mechanisms** could help capture long-term dependencies in speech, improving denoising performance in challenging noise conditions. Additionally, using **hybrid models** that process both time and frequency domains simultaneously can lead to more effective enhancement. Increasing **dataset diversity** with more noise types and using **data augmentation techniques** would make the model more robust. Adopting **multi-task learning**, where the network also performs tasks like speaker identification, could enhance generalization. Using **adversarial training** and **perceptual loss functions** such as PESQ or STOI would improve the perceptual quality of the enhanced speech. Moreover, **Neural Architecture Search (NAS)** can automate the discovery of optimal architectures, leading to better performance. Optimizing for **real-time applications** using techniques like model pruning and quantization can make the system more efficient for embedded devices. Improving **model explainability** with tools like SHAP and ensuring robustness against adversarial attacks can also enhance reliability. Lastly, exploring **cross-domain adaptation** can extend the model's applicability to other domains, such as medical audio or environmental sounds, further broadening its use cases.

11. REFERENCE

1. Park, K.-Y. and H.S. Kim. *Narrowband to wideband conv. of speech using GMM based transformation*. in 2000 IEEE int. conf. on acoustics, speech, and signal processing. Proceedings (Cat. No. 00CH37100). 2000. IEEE.
2. Ramezani, M. and M.M. Homayounpour. *Investigating the effect of phase reconstruction on the BW extention on audio signal using DNNs*. in 2018 9th Int. Symp. on Telecommunications (IST). 2018. IEEE.
3. Makhoul, J. and M. Berouti. *HF regeneration in speech coding systems*. in ICASSP'79. IEEE Inter. Conference on Acoustics, Speech, and Signal Processing. 1979. IEEE.
4. Carl, H. *BW Extension of NB speech signals*. in Proc. EUSIPCO. 1994.
5. Sadasivan, J., S. Mukherjee, and C.S. Seelamantula. *Joint dictionary training for BWE of speech signals*. in 2016 IEEE Inter. Conference on Acoustics, Speech and Signal Processing (ICASSP). 2016. IEEE.
6. Unno, T. and A. McCree. *A robust NB to WB extension system feat. enhanced codebook mapping*. in Proceedings. IEEE Inter. Conference on Acoustics, Speech, and Signal Processing, 2005. 2005. IEEE.
7. Hinton, G., et al., *DNNs for acoustic modeling in speech recognition: The shared views of four research groups*. IEEE Signal processing magazine, 2012. **29**(6): p. 82-97.
8. Li, K. and C.-H. Lee. *A DNN approach to speech BW expansion*. in 2015 IEEE Inter. Conference on Acoustics, Speech and Signal Processing (ICASSP). 2015. IEEE.
9. Yu, H. and W.-P. Zhu. *DNN based complex spectrogram reconstruction for speech BW Expansion*. in 2020 18th IEEE Inter. New Circuits and Systems Conference (NEWCAS). 2020. IEEE.
10. Tagliasacchi, M., et al., *SEANet: A multi-modal SE network*. InterSpeech 2020.

A Joint Bandwidth Expansion and Speech Enhancement Approach using Deep Neural Network

Taieba Taher, Nursadul Mamun, Md. Azad Hossain

Dept. of Electronics and Telecommunication Engineering, Chittagong University of Engineering and Technology, Bangladesh
(taieba.athay; nursad.mamun; azad)@cuet.ac.bd

Abstract— Recently, joint bandwidth expansion and speech enhancement has been a topic of interest in the field of speech processing. The main challenge in this task is to increase the bandwidth of speech signals while enhancing their quality, simultaneously. Deep neural networks (DNNs) have shown great promise in addressing this challenge, as they can learn complex relationships between the input and output signals. In this study, a joint bandwidth expansion and speech enhancement approach using DNNs have been proposed, which is designed to simultaneously increase the bandwidth of speech signals and reduce noise, while preserving speech quality and intelligibility.

This approach leverages the capability of DNNs to simultaneously estimate the missing speech components and the noise profile in the degraded speech signal. The estimated speech components and the noise profile are then used to synthesize a full-band speech signal from a noisy signal with limited bandwidth with improved quality. The network employs three different phases such as oracle, imaged, and noisy phase along with the magnitude spectra to recover high band components. The joint approach demonstrates that the DNNbased bandwidth extension and speech enhancement can be effectively combined to produce high-quality speech signals, outperforms traditional speech enhancement methods, and offers promising solutions for various applications, including speech communication, speech recognition, and speech synthesis.

Keywords—Speech communication, Speech intelligibility, DNN, Speech Enhancement, Bandwidth Extension, Speech Quality.

I. INTRODUCTION

Bandwidth extension is a technique utilized in audio signal processing to enhance the quality of audio signals. This method broadens the frequency range of the original audio signal, resulting in a more vivid and complete sound. In numerous practical applications, the quality of speech signals is impaired by various forms of noise, making speech enhancement a vital task. Bandwidth extension is one solution to enhance the quality of speech signals, as it expands the bandwidth while still maintaining their intelligibility. A joint approach is proposed to mitigate the impact of bandwidth limitations in communication systems, such as telephone networks, and enhance the quality of speech signals for applications like speech recognition and speaker identification. Speech improvement with bandwidth extension may be classified into two approaches: Approaches based on models and approaches based on data. Approaches based on model utilize mathematical models of speech signals to generate new spectral information, while approaches based on data leverage statistical models trained on speech data to estimate and generate new spectral information. Both approaches have their own advantages and limitations, and the appropriate approach is determined based on the specific needs of the application. Several methods have been employed till now to facilitate the retrieval of speech for

979-8-3503-4536-0/23/\$31.00 ©2023 IEEE

recipients [1, 2]. Two primary branches of bandwidth extension have been proposed in [1]. One of these approaches utilizes the only available low-frequency information to extract high frequencies whereas other methods use auxiliary information along with the available low-frequency spectra [2].

Source filtering approaches are utilized to recover the high-frequency range from the residue of low-frequency signal using simple modulation techniques like spectral folding [3] with the estimation of the high-frequency spectral envelope which is rather a challenging task and the excitation created from the low-frequency component. For constructing the connection between high-frequency spectral envelope and low-frequency signals, analysts have effectively put forward several strategies such as codebook mapping [4-6] consolidating with the Gaussian mixture model (GMMs) [1], and the hidden Markov models (HMMs) for classification with codebooks confronted the disadvantage of low learning capacity [2]. Due to the loss of spectral details, the sourcefilter-based method keeps degrading speech quality. Recently, more attention has been given to supervised machine learning to map narrow-band speech to wideband speech for improving speech quality. According to research by Hinton et al. [7], using deep neural networks (DNNs) as the acoustic model for HMMs improves phoneme detection over using GMMs, which inspires DNNs to be employed in the speech processing fields like bandwidth extension as classifiers.

Li and colleagues [8] have demonstrated the application of a DNN to learn the relationship between the log-power magnitude of high-frequency and low-frequency components in order to reconstruct the high spectrum by utilizing a mirror version of the phase. However, mirror phase limits the quality of the expanded speech and yet challenging task to enhance speech due to its unstructured characteristics [9]. Meanwhile, a multi-modal speech denoising network obtains noisy speech by concatenating clean speech with a noise source and feeding it to the multi-modal encoder along with the accelerometer signal [10]. The network shows that it is possible to reconstructed extremely good quality outcomes from band-limited signal, however, difficult to deal with different types of background noise in the band-limited signal. In [11], Radio2Speech has been proposed to analyze RF signals to extract high quality speech. The effectiveness of Radio2Speech is equivalent to the microphone in calm situations, according to experimental findings, and continues to function well in noisy and soundproof environments but shows a bit unsatisfied performance with multi-speaker scenario. Previous research by Y. Gu and colleagues utilized a deep recurrent neural network with long short-term memory (LSTM) cells [12] as a bottleneck feature, combined with a

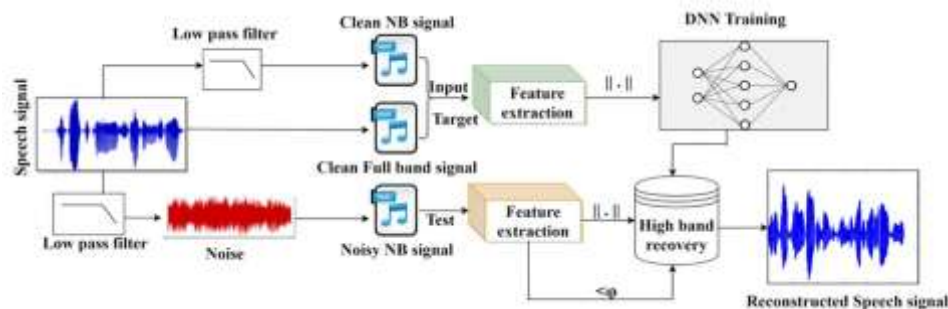


Figure 1. Block diagram of the proposed bandwidth expansion system.

vocoder-based approach using bidirectional input-output mapping [13], for estimating high-frequency spectral components. In this study, we introduce a DNN architecture that can reconstruct clear full-band speech from noisy, bandlimited speech signals by learning the relationship between the log-power magnitude of high-frequency and low-frequency components. The DNN has been framed as a regression problem, allowing it to non-linearly model the clean low-frequency components and predict an approximation of the high-frequency components. Given sufficient training data, DNNs can learn complex models, broadening their use in speech recognition and enhancement [2]. The specific goals of this study are twofold: (i) to reduce noise and produce clean speech from noisy speech, and (ii) to expand the bandwidth and recover the high-frequency spectrum.

The paper is organized as follows. Sec. II explains the methodology of the proposed bandwidth expansion networks. It also includes the feature extraction technique leveraged in the proposed network. Experimental

setup and results of the proposed DNN based network is described in Sec. III. We compare results of the proposed network with results of the traditional methods. Finally, Sec. IV concludes this study.

II. EXPERIMENTAL SETUP

A. Preprocessing

Basic block diagram of the proposed system is shown in Fig.1. The model is divided into two stages: training and testing stage. The input full band audio signal is passed through 20th-order Butterworth filter with a cutoff frequency of 3000 Hz, resulting a narrowband signal. The STFT, $Y(m, n)$ of the signal is then extracted from the output as follows.

$$Y(m, n) = \sum_{k=0}^{N-1} y(m * \Delta + k) h(k) e^{j\phi(k)} \quad (1)$$

Here, m is the frame index, $m = 0, \dots, M-1$ is the frequency index, Δ is the window shift, N is the window length and $h(k)$ is the Hamming window function. Log-spectral power magnitude and phase information has been extracted from STFT as features.

In training stage, the input to the model is STFT magnitude of clean narrowband signal and the target is STFT magnitude of clean full band audio signal. For testing, noisy narrowband is taken as input to recover clean full band signal.

B. Model Description

The deep neural network (DNN) illustrated in Fig.1 consists of an input layer, an output layer, and three dense layers serving as hidden layers, each with 256 nodes. The DNN receives the log spectrum of the narrowband signal as input and outputs the log spectrum of the full-band signal. To standardize the input and output pairs, they were normalized with a zero mean and unit variance across all training samples. The narrowband signal was processed using a window size of 256 with a window shift of 128. The initial learning rate was set to 0.01, and the network was trained using the Adam optimization method. The mean squared logarithmic error (MSLE) loss function was used to measure the difference between the input and target and ensure effective learning by the proposed system. The MSLE loss function can be defined as, $\log^2 \frac{y - \hat{y}}{1 + y^2}$

Here n is the number of input signals, y is a data point of the original signal, and $\hat{y}$ is the data point value of the predicted signal.

C. Signal reconstruction

Even though the precise magnitude of the wideband spectra could be obtained, the phase information was lost. Several algorithms have been developed for synthesizing time domain signal, which necessitates the estimate of phase information for high frequencies whose magnitude has been predicted. The high-frequency spectra have been calculated based on DNN output, where certain energy losses are compensated for the energy loss owing to just half of the points of wideband signal is utilized to construct the narrowband spectrum [3]. For reconstructing phase, three conditions have been considered: “Oracle phase” $\angle Y \angle \hat{Z}$; “Imaged phase” $\angle \hat{Z} \angle \text{flip} \angle X$, and “Noisy phase” $\angle X_{\text{noisy}} \angle \hat{Z}$. After estimating the magnitude of full-band signal, time domain waveform is reconstructed. The inverse short-time Fourier transform (ISTFT) has been performed on $\hat{Z}$, overlapping, and adding with the same hamming window to reconstruct the signal.

$$\hat{Z} \hat{Z}_{\text{mag}} * \exp(j \angle \hat{Z}) \quad 3$$

D. Test corpus

TIMIT dataset consists of broadband recordings of 630 speakers of 4000 samples of 8 major dialect regions of the United States. This study used a part of the dataset (1000 samples from different male and female speakers) for training and 50 samples for testing the network. The testing and training speakers are different and 20% of the training speakers are taken for validation purposes. The speech was upsampled to 16 KHz from its original sampling rate. In this study clean signals are used for training the bandwidth extension model. Noise may be a more prominent thought having the most impact on speech signals. Considering such a fact, both clean and noisy

signals have been used for testing the model to reconstruct the wideband signals. Two types of noises: speech shape noise and car noise are added along with several SNR values with the clean signal to create a noisy environment. We calculated the scores of objective metrics like perceptual evaluation of speech quality (PESQ) [14], short-time objective intelligibility (STOI) [15], and spectrogram orthogonal polynomial measure (SOPM) [16].

III. RESULTS

The performance of the proposed system is described in this section. The performance scores of different objective metrics are shown for a clean and robust condition with two SNRs [5 dB and 10 dB] respectively.

A. Bandwidth expansion for clean condition Table I presents the results of an ablation study conducted on reconstructed signal of the TIMIT corpus in a clean scenario. The study compares the original signal with three different reconstructed signals with different phases (Oracle, Imaged, and Noisy). The results show that the reconstructed signal with the oracle phase has the best performance in terms of both PESQ and SOPM, with a score of 4.44 and 0.98 respectively. The imaged phase and the noisy phase have similar performance with scores of 4.44 and 0.97 for PESQ and 0.98 and 0.96 for SOPM. The STOI results show a slightly lower performance for the noisy phase with a score of 0.96 compared to the oracle and imaged phases, which have a score of 0.98.

Table I. Ablation study on the reconstructed signal with three different phases in clean scenario with TIMIT corpus.

Objective Measures	Original Signal	Reconstructed Signal		
		Oracle phase	Imaged phase	Noisy phase
PESQ	4.42	4.44	4.44	4.44
STOI	0.98	0.98	0.98	0.96
SOPM	0.95	0.98	0.97	0.96

B. Bandwidth expansion for noisy condition Two types of noises are used in this study for assessing the robustness of the proposed system. One is speech-shaped noise (SSN) as full-band noise, and the other is low-band car noise. Table II and Table III show the performance scores of different objective metrics of reconstructed signals in noisy scenario.

Table II. Quantitative evaluation on the reconstructed (Enhanced) signal in a full-band noisy environment.

Objective Measures	10 dB				5 dB			
	Noisy signal	Enhanced signal			Noisy signal	Enhanced signal		
		Oracle	Imaged	Noisy		Oracle	Imaged	Noisy
PESQ	2.30	2.60	2.32	2.32	1.97	2.31	1.97	1.97
STOI	0.86	0.90	0.86	0.87	0.77	0.84	0.79	0.78
SOPM	0.92	0.96	0.93	0.92	0.85	0.90	0.86	0.86

Table II and Table III appear to be the results of an evaluation of audio signal processing. The data is organized into two tables, each presenting result for a different type of noisy environment (full-band or low-band). Each Table presents three objective measures, and these are the standard metrics to evaluate the quality of speech signals. For each measure, the table represents results for four different conditions: the noisy

Table III. Quantitative evaluation on the reconstructed (Enhanced) signal in a low-band noisy environment.

Objective Measures	10 dB				5 dB			
	Noisy signal	Enhanced signal			Noisy signal	Enhanced signal		
		Oracle	Imaged	Noisy		Oracle	Imaged	Noisy
PESQ	2.32	2.60	2.32	2.32	1.97	2.30	1.97	1.97
STOI	0.86	0.90	0.88	0.88	0.76	0.82	0.78	0.77
SOPM	0.91	0.95	0.93	0.923	0.85	0.90	0.86	0.86

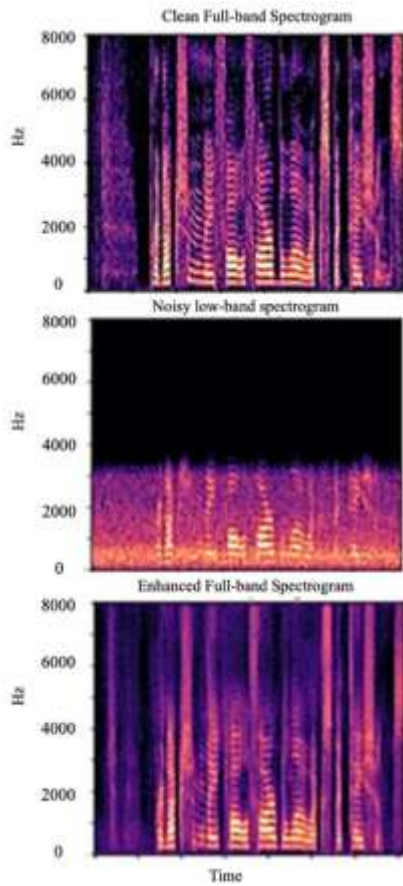


Figure 3. Spectrogram of one test utterance with SNR 10 dB, (a) clean full band signal spectrogram, (b) noisy low band signal spectrogram, (c) enhanced full band signal spectrogram with oracle phase.

signal with low band, an enhanced signal reconstructed by an “oracle” phase, “imaged” phase and “noisy” phase. In general, the results show that the enhanced signal has higher values for each of the three measures, compared to the noisy low band signal. The difference between the enhanced signal and the noisy signal is more pronounced for the full-band noisy environment, where the enhanced signal has improved PESQ, STOI, and SOPM values by 0.3, 0.04, and 0.04 respectively, compared to the noisy signal at 10 dB, and by 0.34, 0.07, and 0.05 respectively at 5 dB. For the low-band noisy environment, the enhanced signal has improved PESQ, STOI, and SOPM values by 0.28, 0.04, and 0.02 respectively, compared to the noisy signal at 10 dB, and by 0.33, 0.06, and 0.05 respectively at 5 dB. Fig. 3 represents the log magnitude spectrogram analysis of one test utterance among 50 test samples. The proposed study does better in reconstructing the high band from the noisy low band without any discontinuity between the low and high band spectrogram. The spectrogram of the enhanced figure is shown in the bottom of Fig. 3.

IV. CONCLUSION

In conclusion, the paper presents a deep neural networkbased approach for joint bandwidth expansion and speech enhancement. The proposed approach aims to increase the bandwidth of speech signals while enhancing their quality and reducing noise. The network employs three different phases along with magnitude spectra to recover high-band components and is designed to simultaneously estimate the missing speech components and the noise profile in the degraded speech signal. The results show that the DNN-based bandwidth extension and speech enhancement approach outperforms in noisy environment. The results of the ablation study and the evaluation of the audio signal processing demonstrate the effectiveness of the enhanced signal compared to the noisy low-band signal. The reconstructed signal with the "oracle" phase performed the best in terms of PESQ and SOPM, with a score of 4.44 and 0.98 respectively. However, the "imaged" and "noisy" phases also showed similar performance. The results were more pronounced in the full-band noisy environment, where the enhanced signal improved PESQ, STOI, and SOPM values by 0.04, 0.03, and 0.01 respectively at 10 dB than at 5 dB. For the low-band noisy environment, the improved values were 0.05, 0.02, and 0.03 respectively at 10 dB than at 5 dB. These

results highlight the potential of the proposed method to effectively improve the quality of speech signals in noisy environments. Overall, the results suggest that the quality of the reconstructed signal is maintained to a certain extent even when the phases are not perfectly accurate.

REFERENCES

1. Park, K.-Y. and H.S. Kim. *Narrowband to wideband conv. of speech using GMM based transformation*. in [2000 IEEE int. conf. on acoustics, speech, and signal processing. Proceedings](#) (Cat. No. 00CH37100). 2000. IEEE.
2. Ramezani, M. and M.M. Homayounpour. *Investigating the effect of phase reconstruction on the BW extention on audio signal using DNNs*. in [2018 9th Int. Symp. on Telecommunications \(IST\)](#). 2018. IEEE.
3. Makhoul, J. and M. Berouti. *HF regeneration in speech coding systems*. in *ICASSP'79. IEEE Inter. Conference on Acoustics, Speech, and Signal Processing*. 1979. IEEE.
4. Carl, H. *BW Extension of NB speech signals*. in *Proc. EUSIPCO*. 1994.
5. Sadasivan, J., S. Mukherjee, and C.S. Seelamantula. *Joint dictionary training for BWE of speech signals*. in [2016 IEEE Inter. Conference on Acoustics, Speech and Signal Processing \(ICASSP\)](#). 2016. IEEE.
6. Unno, T. and A. McCree. *A robust NB to WB extension system feat. enhanced codebook mapping*. in *Proceedings. IEEE Inter. Conference on Acoustics, Speech, and Signal Processing*. 2005. 2005. IEEE.
7. Hinton, G., et al., *DNNs for acoustic modeling in speech recognition: The shared views of four research groups*. [IEEE Signal processing magazine](#), 2012. **29**(6): p. 82-97.
8. Li, K. and C.-H. Lee. *A DNN approach to speech BW expansion*. in [2015 IEEE Inter. Conference on Acoustics, Speech and Signal Processing \(ICASSP\)](#). 2015. IEEE.
9. Yu, H. and W.-P. Zhu. *DNN based complex spectrogram reconstruction for speech BW Expansion*. in [2020 18th IEEE Inter. New Circuits and Systems Conference \(NEWCAS\)](#). 2020. IEEE.
10. Tagliasacchi, M., et al., *SEANet: A multi-modal SE network*. [InterSpeech 2020](#).
11. Zhao, R., et al., *Radio2Speech: High Quality Speech Recovery from RF Signals*. [InterSpeech 2022](#), 11066, 2022.
12. Gu, Y., Z.-H. Ling, and L.-R. Dai. *Speech BWE using bottleneck features and deep RNNs*. in [Interspeech. 2016](#).
13. Gu, Y. and Z.-H. Ling. *Restoring HF spectral envelopes using neural networks for speech BW extension*. in [2015 Inter. Joint Conf. on NN \(IJCNN\)](#). 2015. IEEE.
14. Hao, X., et al. *Time-domain neural network approach for speech BWE*. in [ICASSP 2020-2020 IEEE Inter. Conf. on Acoustics, Speech and Signal Processing \(ICASSP\)](#). 2020. IEEE.
15. Taal, C.H., et al., *An algorithm for intelligibility prediction of time-frequency weighted noisy speech*. [IEEE Transactions on Audio, Speech, and Language Processing](#), 2011. **19**(7): p. 2125-2136.
16. Mamun, N., et al., *An intrusive method for estimating speech intelligibility from noisy and distorted signals*. [The Journal of the Acoustical Society of America](#), 2021. **150**(3): p. 1762-1778.
17. Rix, A.W., et al. *Perceptual evaluation of speech quality (PESQ)-a new method for speech quality assessment of telephone networks and codecs*. in [2001 IEEE international conference on acoustics, speech, and signal processing. Proceedings](#) (Cat. No. 01CH37221). 2001. IEEE.

